Visual Programming

C# (C sharp)

Working with Files

Lecturer: **Dr. Khaled Mohammed BinAbdl**

Department of Information Technology,

College of Computers, Seiyun University

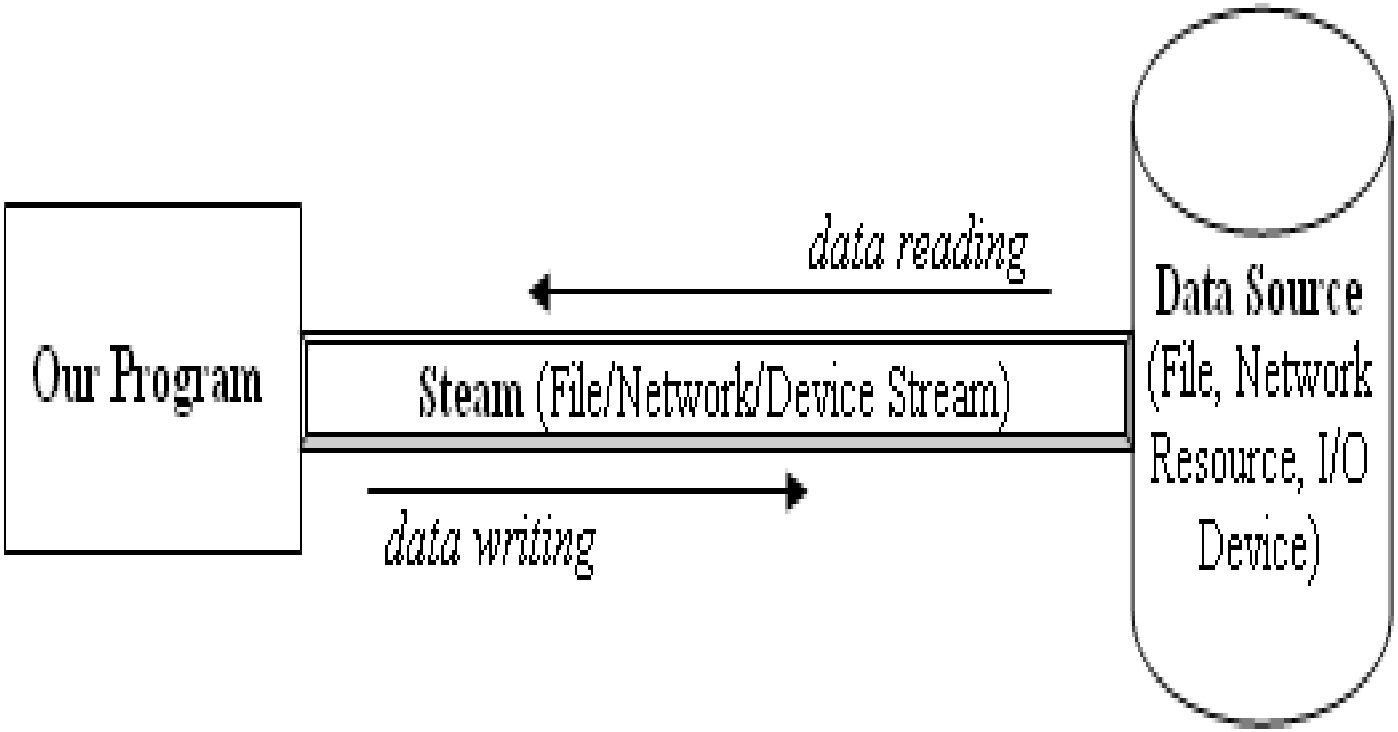
Overview

- Working with File System
- Streams
- Classes Used for I/O
- File Stream
- Stream Reader
- Stream Writer
- Random Access

Streams

- In .Net Framework input and output is done through streams.
- A stream is a continuous flow of bytes from a source to a destination.
- It could be memory to hard disk or from network to loc sys or one area of mem to another.
- An output stream is used when data is written to some external destination.
- Streams provide a way to write and read bytes to and from a storage medium.
- Streams hide the implementation details of input/output operations.
- A data channel having two ends, one is attached to the data source while the other is attached to the reader or writer of the data.

Streams..


Our Program
data reading
Steam (File/Network/Device Stream)
data writing
Data Source
(File, Network
Resource, I/O
Device)

Streams..

- Streams can be used for:
  - **Sequential file access**
    - Read from and write to the beginning of a file
  - **Random file access**
    - Read from and write to any locations within a file

- Streams can be used to perform three fundamental operations:
  - Read from a stream into a data type
  - Write the contents of a data type to a stream
  - Seek particular positions within a stream

Streams..

- Different types of Streams are :-
  - **FileStreams – System.IO.FileStream**
  - **MemoryStreams – System.IO.MemoryStream**
  - **Networkstreams – System.Net.Sockets.NetworkStream**
  - **System.IO.Stream is the base class for all stream classes in IO.**

  - **FileStream – read/write bytes to a file**
  - **BinaryReader and BinaryWriter – To read/write primitive data types**
  - **StreamReader and StreamWriter – To read/write text files**

Streams

- File Modes

  - Enumeration FileMode

  - **Append**
    - **If exists opens and seeks to the end of the file or creates a new file**
  - **Create**
    - **Create a new file, if exists overwrites**
  - **CreateNew**
    - **Create a new file, if exists throws IOException**
  - **Open**
    - **Opens an existing file, else throws FileNotFoundException**
  - **OpenOrCreate**
    - **If exists open or create a new one**
  - **Truncate**
    - **Opens an existing file and truncates to zero bytes**

Streams

- File Access
  - Enum FileAccess
  - Read, ReadWrite, Write
- File Sharing
  - Enum FileShare
  - None – Declines sharing of the current file
  - Read, ReadWrite, Write
- FileMode, FileAccess and FileSharing to be specified in the FileStream constructors

Streams..

- StreamReader and StreamWriter
  - Implements TextReader and TextWriter
- StreamReader
  - String ReadLine()
  - ReadBlock(char[],int index, int count)
  - String ReadToEnd()
- StreamWriter
  - Flush, Write(data type) – Overloaded methods, WriteLine(String)

Classes used for I/0

- File : A utility class that exposes many static methods for moving, copying and del files.
- Directory : A utility class that exposes many static methods for moving, copy and del directories.
- Path : A utility class that is used to manipulate path names.
- FileInfo: Represents phy file on disk, for read and writ a stream obj
- DirectoryInfo : Represents a phy directory on disk.
- FileStream : File that can be written or read or both.
- SteamReader : Read char from a stream.
- StreamWriter : Writes char data to a stream

File & Directory Classes

File Class

- Copy() - Copies the file to specified location
- Create() - Creates a file in a specified path
- Delete() - Deletes a file
- Open() - Returns a FileStream object at the specified path
- Move() - Moves a specified file to specific location

Directory Class

- CreateDirectory() - Creates a directory with the specified path
- Delete() - Deletes the specified directory and all files within it
- GetDirectories() - Returns the array of Directory objects that represent the directories below the current directory
- Move() - Moves a specified directory to the new location
- GetFiles - Returns an array of file objects in the current directory

FileInfo & DirectoryInfo Classes

- Attributes - Gets or Sets the attributes of current file
- CreationTime - Gets the creation date & time for current life
- DirectoryName - Returns the path to the file directory
- Exists - Determine whether file exists
- Fullname - Retrieves the full path of the current file
- Length - Gets the size of file

File Stream Class

- It represents a stream pointing to a file on a disk or network path.
- FileStream class operates on bytes and byte arrays.
- Objects of FileStream class provide random access to files using the Seek() method.
- Seek() method allows read/write positions to be moved within a file.
- General structure of Seek() method is:

  Seek(byte_position, reference_ point)

  Example: Seek(10, SeekOrigin.Begin);

Properties of Seek Origin

- SeekOrigin.Begin – Refers to the beginning of the file position
- SeekOrigin.Current – Refers to the current file position
- SeekOrigin.End – Refers to the end of the file position

File Stream Class Constructor

- Initializes a new instance of the FileStream object for the specified file
- The general structure of the constructor can be given as:

public FileStream(path, FileMode, FileAccess);

In this code,

- path – Represents the file name or file path
- FileMode – Determines how the file is opened or created
- FileAccess – Determines how the file may be accessed by the FileStream object

File Stream and Creating File Stream Object

- **File Mode Property Values**
  - Append
  - Create
  - CreateNew
  - Open
  - OpenOrCreate
- **File Access Property Values**
  - Read
  - Write
  - ReadWrite

FileStream fs= new FileStream(“Myfile.txt”, FileMode.Append, FileAccess. Write);

This opens the file, Myfile.txt , in the append mode for the write operation.

Example – File Stream

```
using System;

using System.IO;

namespace Files
{
    class Class1
    {

        static void Main(string[] args)
        {

            string str;

            try

            {

            FileStream fs =  new FileStream(@"c:\newfile.xls",

FileMode.OpenOrCreate);
```

Example – File Stream

```
StreamWriter sw = new StreamWriter(fs);
    sw.WriteLine("hi");
    sw.WriteLine("Welcome to filehandling classes");
    sw.Close();
}
     catch (IOException e)
        {
 Console.WriteLine("An IO exception Occurred " + e);
        }
```

Example – File Stream

```
StreamReader sr =   new StreamReader(@"c:\newfile.txt");
            str = sr.ReadToEnd();

            while (str != null)
            {
                Console.WriteLine(str);
                str = sr.ReadLine();
            }
            sr.Close();

        }
    }
}
```

Stream Writer

- The objects of StreamWriter class are used to perform write operations to a stream.

Example:

- **StreamWriter sw=new StreamWriter(fs);**

In this code, fs is an object of the stream class.

Example – Stream Writer

```
using System;

using System.IO;

namespace day8
{
    class Class1
    {

        static void Main(string[] args)
        {

            try
            {

FileStream fs =

new FileStream(@"d:\log.txt", FileMode.OpenOrCreate);
```

Example – Stream Writer..

```
StreamWriter sw = new StreamWriter(fs);
sw.WriteLine("hi, how are you");
sw.WriteLine("This is string");
sw.Close();
  }
catch (IOException e)
            {
Console.WriteLine("An IO exception Occurred " + e);
            }
        }
    }
}
```

Stream Reader

- Objects of the StreamReader class is used to perform read operations from a stream.
- Example: StreamReader sr=new StreamReader(fs);
- In the above code, fs is an object of the stream class.

Example – Stream Reader

```
using System;

using System.IO;

namespace day8
{
    class Class1
    {

      static void Main(string[] args)
       {
            string str;

            try
            {

 FileStream fs = new

            FileStream(@"d:\log.txt", FileMode.Open);

  StreamReader sr = new StreamReader(fs);

  str = sr.ReadLine();
```

Example – Stream Reader

```
            while (str != null)
            {
                Console.WriteLine(str);

                str = sr.ReadLine();
            }

            sr.Close();

        }
        catch (IOException e)
        {

         Console.WriteLine("Sorry!.Exception

                Occurred " + e.ToString());


        }
    }
  }
}
```

Random Access

- Reading data using File Stream class is not as easy the StreamReader class because it deals exclusively with Raw bytes.
- Read from and write to any locations within a file is called Random Access.
- This class is used to read files such as images and sounds. For reading strings with FileStream class we have to use conversion classes to convert byte data into character form.

Example – Random Access

```
using System;

using System.IO;

using System.Text;

namespace Files
{
    public class Randomaccess
    {
        static byte[] bdata = new byte[1000];

        static char[] cdata = new char[1000];

        static void writing()
        {
            try
            {

 FileStream fs =  new FileStream(@"c:\rndfile1.txt",
FileMode.OpenOrCreate);


  cdata = "hello how are you??".ToCharArray();
```

Example – Random Access

```
Encoder e = Encoding.UTF8.GetEncoder();
e.GetBytes(cdata, 0, cdata.Length, bdata, 0, true);

 fs.Seek(0, SeekOrigin.Begin);
 fs.Write(bdata, 0, bdata.Length);
 fs.Flush();
          }
catch (IOException e)
          {
  Console.WriteLine("An IO exception occurred" + e);
          }
      }
```

Example – Random Access

```
static void reading()
        {
            try
            {
                FileStream fs =
 new FileStream(@"c:\rndfile1.txt", FileMode.Open);

 //cdata = "hello ".ToCharArray ();

 //e.GetBytes (cdata,0,cdata.Length ,bdata,0,true);

                fs.Seek(0, SeekOrigin.Begin);

                fs.Read(bdata, 0, 10);
            }
            catch (IOException e)
            {
                Console.WriteLine("An IO exception
occurred" + e);
                return;

            }
```

Example – Random Access

```
static void Main()
        {
      //writing();
      //Console.WriteLine("writing completed");

Console.WriteLine("data written in the file:");
            reading();
        }
    }
}
```